ΣΥΣΤΗΜΑΤΑ ΔΙΑΧΕΙΡΙΣΗΣ ΚΑΙ ΑΝΑΛΥΣΗΣ ΔΕΔΟΜΕΝΩΝ

Εαρινό εξάμηνο 2024-2025

Δεύτερη Σειρά Ασκήσεων

Άσκηση 1

1. **Α: Επιλογή στο Cities:**
   - Σύνολο εγγραφών του πίνακα Cities: 5000
   - Σύνολο Χωρών: 100

   Άρα υποθέτοντας ομοιόμορφη κατανομή:
   Τ(Α) = 5000 / 100 = 50

   Β: Επιλογή στο Flights:
   - Σύνολο εγγραφών του πίνακα Flights: 100.000
   - Κατανομή διάρκειας:
     1–2 ώρες: 30%
     3–4 ώρες: 40%
     5–7 ώρες: 15%
     8–10 ώρες: 10%
     10–15 ώρες: 5%

   Η συνθήκη 2 ≤ duration < 6 περιλαμβάνει μέρος από 1–2 ώρες (μόνο οι διάρκειες ίσες με 2), πλήρως 3–4 ώρες, μέρος από 5–7 ώρες (μόνο οι διάρκειες 5)

   Θεωρώντας ομοιόμορφη κατανομή εντός κάθε διαστήματος, τότε:
   Από το διάστημα 1–2 (30%), παίρνουμε μόνο το 2 => 1/2 * 30% = 15%
   Από το διάστημα 3–4 (40%) το παίρνουμε ολόκληρο: 40%
   Από το διάστημα 5–7 (15%), παίρνουμε μόνο το 5 => 1/3 * 15% = 5%
   Άθροισμα: 15% + 40% + 5% = 60%, και άρα:
   Τ(Β) = 60% * 100.000 = 60.000

   Γ: Συνένωση Α και Β μέσω cid = from_cid
   - from_cid είναι ξένο κλειδί προς Cities.cid
   - Έχουμε 50 πόλεις από Ελλάδα (Α)
   - Και 60.000 πτήσεις (Β)

   Υποθέτοντας ομοιόμορφη κατανομή τιμών from_cid σε 5000 πόλεις, η πιθανότητα να έχει μια πτήση from_cid που αντιστοιχεί σε ελληνική πόλη είναι 50/5000 = 1%. Άρα από τις 60.000 πτήσεις, οι αναμενόμενες με from_cid από Ελλάδα είναι:
   Τ(Γ) = 1% * 60.000 = 600

   Δ: Συνένωση με Airlines (μέσω aid)
   Flights έχουν aid ξένο κλειδί σε Airlines (300 εγγραφές)
   Αν υποθέσουμε ομοιόμορφη κατανομή τιμών aid, η σύζευξη είναι many-to-one, άρα δεν αυξάνει τον αριθμό εγγραφών, άρα:
   Τ(Δ) = Τ(Γ) = 600

2. **Επερώτημα**:

SELECT * FROM Flights WHERE fid > 50000
UNION
SELECT * FROM Flights WHERE duration >=2 and duration<6

Η UNION αφαιρεί τα διπλότυπα, οπότε θα χρησιμοποιήσουμε τον τύπο:
$|A \cup B| = |A| + |B| - |A \cap B|$
Δηλαδή:
$T_{UNION} = T_1 + T_2 - T_{1 \cap 2}$
Όπου T_1 είναι οι εγγραφές του πρώτου SELECT, T_2 του δεύτερου και $T_{1 \cap 2}$ της τομής τους.

Για το πρώτο SELECT έχουμε την εξής πιθανότητα:
P(fid>50000) = 100000/50000 = 0.5
Άρα:
$T_1 = 0.5 * 100000 = 50000$

Για το δεύτερο SELECT, όπως έχει αναλυθεί προηγουμένως περιλαμβάνει:
duration = 2 (15%)
duration ∈ [3–4] (40%)
duration = 5 (5%)
Συνολικά: 60% των πτήσεων
Άρα:
$T_2 = 0.6 * 100000 = 60000$

Για τις εγγραφές βρίσκονται και στα δύο σύνολα (τομή), έχουμε την πιθανότητα:
P(fid>50000 AND 2≤duration<6) = 0.5 * 0.6 = 0.3
Άρα:
$T_{1 \cap 2} = 0.3 * 100000 = 30000$

Τελικά το αποτέλεσμα της UNION είναι:
$T_{UNION} = T_1 + T_2 - T_{1 \cap 2} = 50000 + 60000 - 30000 \Rightarrow$ **$T_{UNION} = 80.000$**

Άσκηση 2

1. **Βέλτιστο Κόστος SMJ για B(R1) = 600, B(R2) = 400, M = 5**

Ο αλγόριθμος SMJ θα δημιουργήσει τις εξής ταξινομημένες λίστες για την σχέση R1:

- Στο πρώτο πέρασμα θα διαβάσει τη σχέση R1 και θα γράψει 120 ταξινομημένες λίστες μεγέθους 5 σελίδων η καθεμία. **Κόστος= 2*B(R1)**.
- Στο δεύτερο πέρασμα θα διαβάσει τις 120 ταξινομημένες λίστες και θα τις συγχωνεύσει σε 24 ταξινομημένες λίστες των 25 σελίδων η καθεμία. **Κόστος= 2*B(R1).**
- Στο τρίτο πέρασμα θα διαβάσει τις 25 λίστες και θα τις συγχωνεύσει σε 5 ταξινομημένες λίστες των 120 σελίδων η καθεμία. **Κόστος= 2*B(R1).**

- Στο τέταρτο πέρασμα θα διαβάσει τις 5 λίστες και θα τις συγχωνεύσει σε 1 ταξινομημένη λίστα των 600 σελίδων. **Κόστος= 2*B(R1).**
-

Ο αλγόριθμος SMJ θα δημιουργήσει τις εξής ταξινομημένες λίστες για την σχέση R2:

- Στο πρώτο πέρασμα θα διαβάσει την σχέση R2 και θα γράψει 80 ταξινομημένες λίστες των 50 σελίδων η καθεμία. **Κόστος= 2*B(R2).**
- Στο δεύτερο πέρασμα θα διαβάσει τις 80 ταξινομημένες λίστες και θα τις συγχωνεύσει σε 16 των 25 σελίδων η καθεμία. **Κόστος= 2*B(R2).**
- Στο τρίτο πέρασμα θα **διαβάσει** τις 16 λίστες και θα τις συγχωνεύσει σε 4 λίστες των 100 σελίδων η καθεμία. **Κόστος= 2*B(R2).**

Στο σημείο αυτό ο αλγόριθμος SMJ έχει δημιουργήσει στον δίσκο **1** ταξινομημένη λίστα για τη σχέση R1 και **4** ταξινομημένες λίστες για την σχέση R2. Η μνήμη χωράει 5 σελίδες οπότε ο αλγόριθμος SMJ θα προχωρήσει στην συγχώνευση των λιστών και θα δώσει το αποτέλεσμα στην έξοδο.
Το κόστος του output δεν προσμετράται. Άρα **Κόστος=B(R1)+B(R2)**

Συνολικό Κόστος = 9*B(R1) + 7 * B(R2) = 9 * 600 + 7 * 400 = 5400 + 2800 =>
Συνολικό Κόστος = 8200 I/O

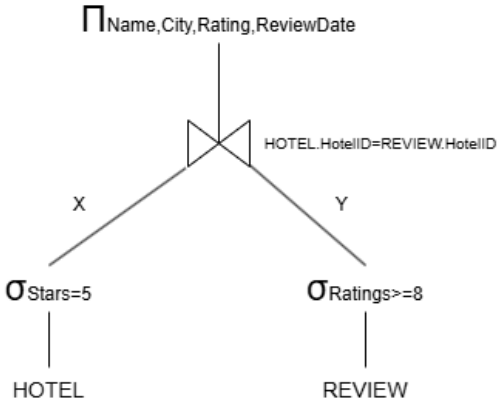
2. Στόχος είναι να **ταξινομηθεί** η κάθε σχέση **ως προς το γνώρισμα b**, αφού η σύζευξη γίνεται στη συνθήκη R1.b=R2.b

   Έτσι δημιουργούμε clustered B+Tree ευρετήρια στα γνωρίσματα R1.b και R2.b. Αυτή η επιλογή εγγυάται ότι τα δεδομένα των δύο σχέσεων είναι αποθηκευμένα **σε φυσική ταξινόμηση** ως προς b, κάνοντας την εκτέλεση του SMJ ιδανικά αποδοτική. Έχουμε 1 ταξινομημένη λίστα για την R(1) και μία για την R(2), οπότε ο αλγόριθμος SMJ θα προχωρήσει στην συγχώνευση των λιστών και θα δώσει το αποτέλεσμα στην έξοδο. Το κόστος του output δεν προσμετράται.

   Κόστος = B(R1) + B(R2) = 600 + 400 = 1000 I/O

Άσκηση 3

Α. Καθώς οι επιλογές πρέπει να εφαρμόζονται πριν τις συζεύξεις, ώστε να μειωθεί το μέγεθος των ενδιαμέσων σχέσεων και συνεπώς το κόστος, προκύπτει το παρακάτω βελτιστοποιημένο λογικό πλάνο:



Β. Από τα δεδομένα της άσκησης έχουμε:

- T(HOTEL) = 20.000
- B(HOTEL) = 20.000/50 = 400
- T(REVIEW) = 400.000
- B(REVIEW) = 400.000/500 = 800

Τα ξενοδοχεία με Stars=5 είναι το 10% του συνόλου των ξενοδοχείων, άρα 2000.

Τα ξενοδοχεία με Rating>=8 είναι το 30% του συνόλου γιατί οι τιμές του πεδίου Rating κατανέμονται ομοιόμορφα, άρα 120.000.

Έστω X = $\sigma_{(Stars=5)}$HOTEL και Y = $\sigma_{(Rating>=8)}$REVIEW, τότε με βάση τα παραπάνω, έχουμε:

- T(X) = 2000
- B(X) = 2000/50 = 40
- T(Y) = 120.00
- B(Y) = 120.000/500 = 240

Το κόστος υπολογισμού του X με table scan είναι B(HOTEL)=400, ενώ με χρήση του απλού (non clustered) ευρετηρίου είναι T(X)=2000. Οπότε επιλέγουμε το μικρότερο κόστος, έτσι **cost(X)=400**

Το κόστος υπολογισμού του Y με table scan είναι B(REVIEW)=800, ενώ με χρήση του clustered ευρετηρίου είναι B(Y)=240 καθώς οι σελίδες στον δίσκο είναι συνεχόμενες. Οπότε επιλέγουμε το μικρότερο κόστος, έτσι **cost(Y)=240**

<u>Κόστος SMJ</u>

Για τον υπολογισμό του κόστους στον SMJ θα προσθέσουμε το κόστος των επιλογών (cost(X)+cost(Y)), τα κόστη της ταξινόμησης (2B(X)+2B(Y)) και το κόστος της συνένωσης (merge join) (B(X)+B(Y)). Άρα:

cost(SMJ) = cost(X) + cost(Y) + 2*B(X) + 2*B(Y) + B(X) + B(Y) =
= 400 + 240 + 2*40 + 2*240 + 40 + 240 =>
cost(SMJ)= 1480 I/O

<u>Κόστος BNLI με εξωτερική την σχέση X</u>

Κάνουμε την X εξωτερική σχέση γιατί έχει μικρότερο πλήθος μετά την επιλογή. Για τον υπολογισμό του κόστους στον BNLI θα προσθέσουμε:

- το κόστος των επιλογών: Το κόστος επιλογής για HOTEL είναι **400** I/Os και το κόστος επιλογής για REVIEW είναι **240**
- το κόστος της συνένωσης (merge join): B(X) + ⌈B(X) / M-1⌉ * B(Y) = 40 + ⌈40/20⌉·240 = 40 + 2 * 240 = **520**

Συνεπώς:

cost(BNLI) = 400 + 240 + 520 = 1160 I/O

Άσκηση 4

Από την εκφώνηση έχουμε ότι η σχέση ΥΠΑΛΛΗΛΟΣ καταλαμβάνει 10.000 blocs και το 10% των πλειάδων της σχέσης ικανοποιούν την συνθήκη Ειδικότητα = 'Ηλεκτρολόγος', δηλαδή **1000** blocs.

Επίσης το 10% των πλειάδων ικανοποιούν την συνθήκη Τμήμα = 'Παραγωγή', δηλαδή **1000** blocs, ενώ και τις δύο συνθήκες ταυτόχρονα τις ικανοποιεί ένα 5% των πλειάδων δηλαδή **500** blocs.

1. **Ευρετήριο συστάδων (clustering index) στο γνώρισμα Ειδικότητα:**

   Εφόσον το ευρετήριο για την Ειδικότητα βρίσκεται στην μνήμη, το index seek δεν προσμετράται, οπότε θα προσπελάσουμε 1000 blocs (όλα τα blocks με Ειδικότητα = 'Ηλεκτρολόγος') και στην συνέχεια για κάθε πλειάδα που ανακτήσαμε, φιλτράρουμε στη μνήμη το Τμήμα = 'Παραγωγή'. Το clustered index εγγυάται ότι όλες οι εγγραφές που ικανοποιούν τη συνθήκη θα βρίσκονται συνεχόμενα στον δίσκο (ή σχεδόν). Άρα το κόστος είναι, 1 για να διαβάσουμε το φύλλο του index και 1000 για να διαβάσουμε τα block δεδομένων:
   cost = 1 + 1000 = 1001 I/O

2. **Ευρετήριο συστάδων (clustering index) στο ζεύγος γνωρισμάτων (Ειδικότητα, Τμήμα) με την σειρά που δίνονται:**

   Τα δεδομένα είναι ταξινομημένα πρώτα κατά Ειδικότητα και μετά κατά Τμήμα. Το ευρετήριο μπορεί να χρησιμοποιηθεί για να αναζητηθούν συγκεκριμένες τιμές και στα δύο γνωρίσματα, δηλαδή ακριβώς η μορφή που ζητάει η επερώτηση. Στην περίπτωση αυτή δεν χρειάζεται φιλτράρισμα in-memory (το index εφαρμόζεται πλήρως). Το clustered index εγγυάται ότι όλες οι εγγραφές που ικανοποιούν τη συνθήκη θα βρίσκονται συνεχόμενα στον δίσκο (ή σχεδόν), οπότε θα γίνει προσπέλαση μόνο των σελίδων δεδομένων που περιέχουν τα αποτελέσματα, δηλαδή το κόστος είναι, 1 για να διαβάσουμε το φύλλο του index και 500 για να διαβάσουμε τα block δεδομένων:
   cost = 1 + 500 = 501 I/O

3. **Ευρετήριο συστάδων (clustering index) στα ακόλουθα τρία γνωρίσματα (Τμήμα, Ειδικότητα, Επώνυμο) με την σειρά που δίνονται:**

   Τα δεδομένα είναι ταξινομημένα πρώτα κατά Τμήμα μετά κατά Ειδικότητα και τέλος κατά Επώνυμο. Ξεκινώντας από το Τμήμα, μπορεί να προσπελάσει τις εγγραφές με Τμήμα='Παραγωγή'. Στην συνέχεια φιλτράρει Ειδικότητα = 'Ηλεκτρολόγος' μέσα στο εύρος των εγγραφών με Τμήμα = 'Παραγωγή'. Το ευρετήριο καλύπτει πλήρως την σύνθετη συνθήκη. Το clustered index εγγυάται ότι όλες οι εγγραφές που ικανοποιούν τη συνθήκη θα βρίσκονται συνεχόμενα στον δίσκο (ή σχεδόν), οπότε θα γίνει προσπέλαση μόνο των σελίδων δεδομένων που περιέχουν τα αποτελέσματα, δηλαδή όπως και προηγουμένως:
   cost = 1 + 500 = 501 I/O

4. **Ευρετήριο συστάδων (clustering index) στα ακόλουθα τρία γνωρίσματα (Επώνυμο, Ειδικότητα, Τμήμα) με την σειρά που δίνονται:**

   Τα δεδομένα είναι ταξινομημένα πρώτα κατά Επώνυμο μετά κατά Ειδικότητα και τέλος κατά Τμήμα. Ξεκινά από το Επώνυμο για το οποίο δεν υπάρχει συνθήκη. Άρα δεν μπορεί να χρησιμοποιηθεί αποδοτικά, με αποτέλεσμα να γίνει πλήρης σάρωση όλων των εγγραφών της βάσης για να εκτελεστεί η επερώτηση. Δηλαδή:
   cost = 10.000 I/O